Optymalizacja Sieci Lab

Dawid Nowak
Jan Narożny
Adrian Czubaty

Czerwiec 2026

1 Wstęp

Sprawozdanie przygotowano w ramach kursu "Optymalizacja systemów i sieci informatycznych nowej generacji"realizowanego w trakcie semestru letniego 2025/26 na Politechnice Wrocławskiej. Tematem sprawozdania jest rozwiązanie problemu rozprzestrzeniania danych w sieci informatycznej świadomej przesyłanych treści. Rozpatrzono rozwiązanie wykorzystujące solver oraz implementację algorytmu zaproponowanego w artykule [1].

2 Opis problemu

W rozwiązywanym problemie mamy do czynienia z siecią świadomą treści. Problem modelowany jest jako graf sieciowy, w którym istnieją:

- wierzchołek źródła (publisher)
- wierzchołki serwerów (servers)
- wierzchołki klientów (clients)
- łącza (links)

Dostępna jest macierz routing'u, która opisuje, które łącza l uczestniczą w przesyłaniu obiektu n między klientem (lub źródłem) m a serwerem s. Tak skonstruowany graf uniemożliwia proste przedstawienie graficzne. Problem zakłada nieskończone pojemności serwerów.

Problem w takiej postaci łączy problem dyskretny z problemem nieliniowym, tworząc złożone zagadnienie.

2.1 Zmienne decyzyjne

Problem składa się z dwóch zmiennych decyzyjnych:

1. x_{mns} - wektor przypisujący dla użytkownika m do pobierania obiektu n serwer s. Decyduje on, co skąd jest pobierane.

2. y_{mns} - wektor przypisujący przepustowość dla użytkownika m do pobierania obiektu n z serwera s.

2.2 Dane wejściowe

Danymi wejściowymi są:

- M - liczba klientów
- N - liczba obiektów
- S - liczba serwerów
- L - liczba łączy
- A - macierz routing'u
- C_l - wektor pojemności łączy
- b_n - wektor rozmiarów obiektów

2.3 Ograniczenia

W danym problemie można zdefiniować poniższe ograniczenia:

1. Każdy użytkownik posiada dla każdego zasobu przypisany serwer

$$\forall_{m\in\{1,\ldots,M\}}\forall_{n=1,\ldots,N}\ \sum_{s=1}^{S} x_{mns} = 1$$

2. Obiekt musi być dostępny na serwerze, do którego przypisano pobieranie użytkownikowi

$$\forall_{m\in\{1,\ldots,M\}}\forall_n\forall_s \quad x_{mns} \leq x_{0ns}$$

3. Pojemność łączy nie zostaje przekroczona

$$\forall_{l=1,\ldots,L} \quad \sum_{m=0}^{M}\sum_{n=1}^{N}\sum_{s=1}^{S} a_{mnsl}x_{mns}y_{mns} \leq C_l$$

2.4 Funkcja celu i model opóźnień

Celem operatora sieci jest minimalizacja sumy średnich opóźnień postrzeganych przez wszystkich użytkowników (klientów i wydawców). Model opóźnień przyjęto zgodnie z pracą [1], na której oparto cały algorytm.

Całkowite opóźnienie żądania użytkownika m dotyczącego obiektu n pobieranego z serwera s oznaczono przez $\tau_{mns}(x, y)$ i wyraża się ono wzorem

$$\tau_{mns}(x,y) = \tau^{(1)}_{mns}(y_{mns}) + \sum_{l=1}^{L} a_{mnsl}\Big(\tau^{(2)}_l(x,y) + \tau^{(3)}_l\Big),$$

gdzie poszczególne składniki oznaczają:

- $\tau^{(1)}_{mns}$ — czas wepchnięcia (push) wszystkich bitów obiektu n na łącze przez węzeł źródłowy:
$$\tau^{(1)}_{mns}(y_{mns}) = \frac{b_n}{y_{mns}}.$$
W rozważanej implementacji cały obiekt traktowany jest jako pojedynczy transfer (współczynnik $\alpha_{mns} = 1$ z oryginalnego sformułowania);

- $\tau^{(2)}_l$ — średni czas przebywania pakietu w kolejce rutera na łączu l. Przy założeniu, że kolejki w ruterach są typu M/M/1, składnik ten jest odwrotnością nadwyżki pojemności łącza nad jego obciążeniem:
$$\tau^{(2)}_l(x,y) = \frac{1}{C_l - \sum_{m=0}^{M}\sum_{n=1}^{N}\sum_{s=1}^{S} a_{mnsl}\,x_{mns}\,y_{mns}}.$$
Założenie M/M/1 jest uzasadnione dla ruchu typu FTP (statycznego transferu plików), który dominuje w sieciach świadomych treści;

- $\tau^{(3)}_l$ — suma stałych opóźnień łącza l (propagacja, przetwarzanie), niezależnych od obciążenia.

Funkcja celu jest sumą opóźnień wszystkich aktywnych przepływów:

$$Q(x,y) = \sum_{m=0}^{M}\sum_{n=1}^{N}\sum_{s=1}^{S} x_{mns}\,\tau_{mns}(x,y).$$

Pełne sformułowanie problemu polega na wyznaczeniu

$$(x^*, y^*) = \arg\min_{x,y} Q(x,y)$$

przy spełnieniu ograniczeń 1.–3. Jest to nieliniowy problem programowania mieszanego (dyskretno-ciągły), NP-trudny już przy ustalonym y, co motywuje zastosowanie heurystyki opisanej w dalszej części.

3 Algorytm rozwiązania

Wyjściowy problem łączy dwie klasy problemów - problem dyskretny oraz nieliniowy. Próba rozwiązania obu jednocześnie jest zadaniem wyjątkowo złożonym. Opracowany algorytm stosuje heurystykę, która rozdziela problem z powrotem na dwie osobne klasy i poprzez naprzemienne rozwiązywanie odpowiadających im problemów dąży do uzyskania rozwiązania. Algorytm został stworzony z myślą o wersjach rozproszonej i scentralizowanej.

3.1 Podproblem wyższego poziomu

Poszukiwanie rozwiązania dla problemu optymalnego rozkładu przepływów łączy przy ustalonej topologii sieci, tj. x przestaje być zmienną decyzyjną, a zostaje podane na wejściu jako x'. Podlega pod 3. ograniczenie.

$$y' = arg\ min \sum_{m=2}^{M} \sum_{n=1}^{N} \sum_{s=1}^{S} x'_{mns} \tau_{mns}(x', y)$$

3.2 Podproblem niższego poziomu

Poszukiwanie rozwiązania dla problemu optymalnego przypisania zasobów. Na wejściu podane są liczby klientów, serwerów, zasobów oraz wektor kosztów pobrania obiektu n przez klienta m z serwera s: $\bar{\tau} = [\bar{\tau}_{mns}]$. Podlega on pod ograniczenia 1. oraz 2.

$$x' = arg\ min \sum_{m=0}^{M} \sum_{n=1}^{N} \sum_{s=1}^{S} x_{mns} \bar{\tau}_{mns}$$

3.3 Wykorzystanie heurystyk

Heurystyki są od siebie zależne, przez co naprzemienne ich obliczanie prowadzi do zmiany wyniku. Działanie to wykorzystywane jest dopóki wartość funkcji celu maleje. Do ich rozwiązania wykorzystać można dowolny znany algorytm. Nie gwarantuje to jednak uzyskania globalnego ekstremum.

3.4 Inicjalizacja

Ponieważ obie heurystyki polegają na danych z poprzedniego wykonania, potrzebne jest wykonanie początkowe. W tym celu konstruowane jest proste rozwiązanie dopuszczalne, które ustala wyłącznie zmienną przypisania x. Przyjmuje się, że wszystkie obiekty zostają umieszczone na pierwszym serwerze ($s = 0$):

$$\forall_{n=1,\ldots,N} \quad x_{0n0} = 1, \qquad x_{0ns} = 0 \text{ dla } s \neq 0,$$

a każdy klient pobiera każdy obiekt z tego samego serwera:

$$\forall_{m=1,\ldots,M} \forall_{n=1,\ldots,N} \quad x_{mn0} = 1, \qquad x_{mns} = 0 \text{ dla } s \neq 0.$$

Tak dobrane przypisanie spełnia ograniczenia 1. oraz 2. niezależnie od danych wejściowych (każdy klient ma dokładnie jeden przypisany serwer, a obiekt jest na nim dostępny, ponieważ został tam umieszczony przez wydawcę), stanowi więc poprawny punkt startowy. Na jego podstawie w pierwszej iteracji pętli zewnętrznej rozwiązywany jest najpierw podproblem wyższego poziomu (alokacja przepustowości y przy ustalonym x). Otrzymane wartości y wraz z przyjętym x wyznaczają następnie wektor kosztów $\bar{\tau}$ używany przez podproblem niższego poziomu, co umożliwia rozpoczęcie naprzemiennego rozwiązywania obu heurystyk.

4 Wyniki implementacji

4.1 Dane wejściowe

Algorytm przetestowano na instancji testowej `data1.json`. Sieć składa się z $M = 1$ klienta, $N = 2$ obiektów, $S = 2$ serwerów oraz $L = 4$ łączy. Rozmiary obiektów wynoszą $b_0 = 5{,}0$ oraz $b_1 = 6{,}0$, natomiast pojemności łączy to $C_0 = 20$, $C_1 = 25$, $C_2 = 20$, $C_3 = 20$ (w jednostkach przepustowości).

Macierz routingu A opisuje, które łącza uczestniczą w każdej możliwej ścieżce. Tabela 1 przedstawia rozkład łączy dla każdej kombinacji (m, n, s), gdzie $m = 0$ oznacza wydawcę (publisher), a $m = 1$ klienta.

Tabela 1: Łącza wykorzystywane na poszczególnych trasach (macierz A)

m	n	s	l_0	l_1	l_2	l_3
publisher	0	0		+		+
publisher	0	1	+		+	
publisher	1	0			+	+
publisher	1	1		+		+
klient 0	0	0	+			+
klient 0	0	1		+	+	
klient 0	1	0		+	+	+
klient 0	1	1	+			+

4.2 Przebieg algorytmu i uzyskane rozwiązanie

Algorytm wykonał **2 iteracje** zewnętrznej pętli (na maksymalne 10 dozwolonych) i zakończył działanie po wykryciu stabilizacji przypisania x. W każdej iteracji podproblem górnego poziomu (alokacja przepływów) był rozwiązywany metodą SLSQP, która każdorazowo zwróciła rozwiązanie dopuszczalne (`rate_ok` = True).

Uzyskane rozwiązanie końcowe osiąga wartość funkcji celu $Q = 4{,}292$.

Rozmieszczenie obiektów. Algorytm zdecydował o umieszczeniu obu obiektów na serwerze $s = 1$:

Tabela 2: Historia wartości funkcji celu w kolejnych iteracjach

Iteracja	Q
1	6,82138
2	4,29156

- obiekt 0: serwer 1, szybkość uploadu $y_{0,1} = 6{,}68$,
- obiekt 1: serwer 1, szybkość uploadu $y_{1,1} = 7{,}87$.

Przypisanie klientów. Jedyny klient ($m = 0$) pobiera oba obiekty z serwera 1:

- obiekt 0: szybkość $y_{0,0,1} = 7{,}67$,
- obiekt 1: szybkość $y_{0,1,1} = 6{,}96$.

Obciążenie łączy. Tabela przedstawia stan łączy w uzyskanym rozwiązaniu. Żadne z łączy nie przekracza swojej pojemności maksymalne wykorzystanie wynosi 74,1%.

Tabela 3: Obciążenie łączy w uzyskanym rozwiązaniu

Łącze l	Obciążenie	Pojemność C_l	Utilizacja	Opóźnienie kolejkowe
0	13,636	20	68,2%	0,1571
1	15,537	25	62,1%	0,1057
2	14,350	20	71,8%	0,1770
3	14,823	20	74,1%	0,1932

5 Porównanie rozwiązań

W celu porównania przygotowanych rozwiązań uruchomiono programy dla identycznych danych wejściowych. Zestawiono podejście oparte na solverze optymalizacyjnym oraz implementację algorytmu heurystycznego opisanego wcześniej. Porównanie przeprowadzono pod kątem wartości funkcji celu, dopuszczalności rozwiązania, obciążenia łączy, rozmieszczenia obiektów oraz czasu działania.

5.1 Dane wejściowe

Oba rozwiązania uruchomiono dla tej samej instancji testowej `data1.json`, zawierającej jednego klienta, dwa obiekty, dwa serwery oraz cztery łącza. Dzięki temu możliwe było bezpośrednie porównanie wyników bez wpływu różnic w topologii lub parametrach wejściowych.

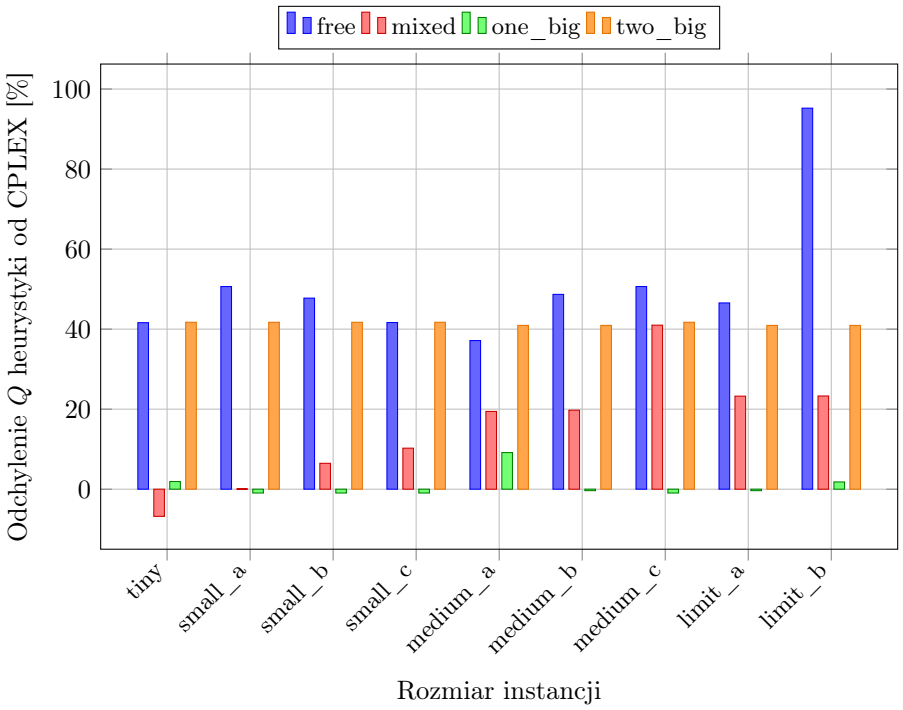
5.2 Wyniki

Aby zbadać zachowanie obu metod w różnych warunkach sieciowych, przygotowano zestaw instancji testowych podzielonych na cztery rodziny, różniące się *profilem pojemności łączy* (C_l). We wszystkich instancjach rozmiary obiektów są jednakowe ($b_n = 1$), dzięki czemu obserwowane różnice wynikają wyłącznie z topologii przepustowości:

- `free` — wszystkie łącza mają dużą, jednakową przepustowość ($C_l = 100$). Sieć jest praktycznie nieograniczona, brak wyraźnego wąskiego gardła;
- `mixed` — łącza mają zróżnicowane przepustowości (np. $C = [5, 20, 10, 35, 8, 15]$), co odwzorowuje sieć niejednorodną;
- `one_big` — tylko jedno wspólne łącze serwera ma dużą przepustowość ($C = 100$), pozostałe są wąskie ($C = 1$). Wymusza to skupienie ruchu na jednej preferowanej ścieżce;
- `two_big` — dwa wspólne łącza serwerów mają dużą przepustowość, pozostałe są wąskie. Dostępne są dwie alternatywne ścieżki o wysokiej przepustowości.

Każda rodzina obejmuje 10 instancji o rosnącym rozmiarze — od najmniejszej (`tiny`: $M = 1$, $N = 2$, $S = 2$, $L = 6$), przez warianty `small`, `medium` i `limit`, aż po instancję `too_large` ($M = 4$, $N = 4$, $S = 3$, $L = 18$). Pozwala to ocenić zarówno jakość rozwiązania, jak i skalowalność obu metod. Wyniki działania obu metod zestawiono w tabeli (kolejne bloki odpowiadają rodzinom `free`, `mixed`, `one_big` oraz `two_big`). Wartości `inf` oznaczają instancje `too_large`, których nie udało się rozwiązać solverem CPLEX: po linearyzacji odcinkowo-liniowej model tych instancji przekracza limit liczby zmiennych narzucany przez darmową edycję CPLEX Community Edition (1000 zmiennych i 1000 ograniczeń), wobec czego solver odmawia jego uruchomienia. Heurystyka, niepodlegająca takim ograniczeniom, rozwiązuje te instancje bez przeszkód.

M	N	S	L	CPLEX Q	heurystyka Q	% różnicy	CPLEX czas [s]	heur czas [s]
2	2	2	8	5.4248	7.9246	46.0803	0.4953	0.1771
1	2	2	4	4.5416	4.2916	-5.5055	0.1501	0.1765
1	2	2	6	0.2964	0.4197	41.6118	0.0969	0.1485
2	2	2	8	0.5260	0.7925	50.6441	0.1937	0.2128
3	2	2	10	0.8600	1.2706	47.7493	0.5334	0.3970
4	2	2	12	1.3063	1.8504	41.6451	0.4579	0.5574
2	3	2	8	1.1375	1.5599	37.1373	0.4011	0.3073
3	3	2	10	1.7084	2.5402	48.6864	0.6856	0.6126
2	2	3	12	0.5260	0.7925	50.6441	0.3012	0.3626
4	3	2	12	2.5523	3.7402	46.5454	2.8390	0.9669
3	3	3	15	1.3012	2.5402	95.2215	5.4341	0.8907
4	4	3	18	-	6.22791	-inf	inf	2.2231
1	2	2	6	2.7891	2.5997	-6.7920	0.1398	0.1037
2	2	2	8	4.9841	4.9877	0.0725	0.3971	0.2356
3	2	2	10	8.3016	8.8384	6.4669	0.3655	0.5486
4	2	2	12	11.5431	12.7265	10.2524	2.6381	0.7561
2	3	2	8	8.3269	9.9454	19.4371	0.7775	0.4248
3	3	2	10	14.6666	17.5622	19.7425	1.8182	0.9705
2	2	3	12	3.1696	4.4690	40.9928	0.6997	0.6990
4	3	2	12	20.7440	25.5663	23.2464	8.8797	1.6177
3	3	3	15	8.9171	10.9940	23.2906	17.3417	1.8866
4	4	3	18	-	22.98074	-inf	inf	3.8434
1	2	2	6	22.9149	23.3542	1.9170	0.2263	0.1466
2	2	2	8	35.3720	35.0316	-0.9621	0.4009	0.4405
3	2	2	10	47.1631	46.7093	-0.9622	0.7772	0.6146
4	2	2	12	58.9546	58.3873	-0.9622	3.7274	1.3830
2	3	2	8	61.6306	67.2687	9.1481	1.2659	0.6487
3	3	2	10	90.0221	89.6923	-0.3663	3.2560	1.4704
2	2	3	12	35.3720	35.0316	-0.9621	1.0841	0.5041
4	3	2	12	112.5286	112.1164	-0.3663	3.5469	2.5949
3	3	3	15	88.0969	89.6923	1.8110	4.3115	2.2705
4	4	3	18	-	180.20690	-inf	inf	4.8865
1	2	2	6	16.4795	23.3542	41.7167	0.2825	0.2131
2	2	2	8	24.7196	35.0316	41.7161	0.2925	0.4236
3	2	2	10	32.9599	46.7093	41.7155	0.3535	0.9001
4	2	2	12	41.2005	58.3873	41.7149	0.5878	1.1772
2	3	2	8	47.7317	67.2687	40.9307	0.3639	0.5147
3	3	2	10	63.6431	89.6923	40.9302	2.6115	1.4350
2	2	3	12	24.7196	35.0316	41.7161	0.4373	0.5138
4	3	2	12	79.5548	112.1164	40.9298	13.0158	3.0651
3	3	3	15	63.6431	89.6923	40.9302	7.9623	1.7757
4	4	3	18	-	180.20690	-inf	inf	6.5981



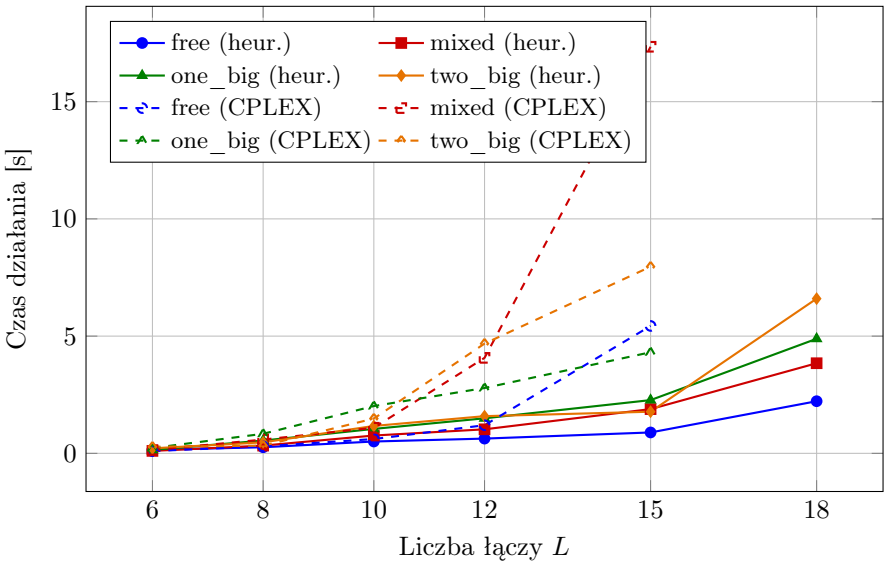
Rysunek 1: Względne odchylenie wartości funkcji celu uzyskanej przez heurystykę od rozwiązania CPLEX (kolumna *% różnicy*) dla każdej rodziny danych. Wartości dodatnie oznaczają rozwiązanie gorsze od solvera, ujemne — lepsze (heurystyka oceniana na dokładnej postaci Q). Pominięto instancje `too_large` ($L = 18$), dla których CPLEX nie zwrócił rozwiązania.

5.3 Podsumowanie porównania

Przeprowadzone eksperymenty pozwoliły porównać jakość rozwiązań uzyskiwanych przez solver CPLEX oraz zaimplementowaną heurystykę. Dla wszystkich analizowanych instancji heurystyka zwróciła rozwiązanie dopuszczalne, spełniające ograniczenia pojemności łączy. Względną jakość obu metod podsumowano na rysunku 1, a porównanie czasów działania na rysunku 2.

W przypadku sieci o równomiernych przepustowościach (`free`) heurystyka uzyskiwała wyniki gorsze od rozwiązania optymalnego wyznaczanego przez CPLEX. Jak widać na rysunku 1, różnica wartości funkcji celu wynosiła zazwyczaj od około 40% do 50%. Oznacza to, że w sytuacji braku wyraźnych ograniczeń sieciowych metoda heurystyczna nie była w stanie znaleźć rozwiązania o jakości porównywalnej z optimum.

Znacznie lepsze rezultaty uzyskano dla instancji `mixed` oraz `one_big`. W tych przypadkach różnice pomiędzy heurystyką a rozwiązaniem solvera były niewielkie i często nie przekraczały kilku procent. Dla części instancji heurystyka



Rysunek 2: Czas działania heurystyki (linie ciągłe) i solvera CPLEX (linie przerywane) w funkcji liczby łączy L, dla czterech rodzin danych wejściowych. Dla instancji o tej samej wartości L czasy uśredniono. CPLEX nie zwrócił rozwiązania dla największych instancji ($L = 18$), stąd brak odpowiednich punktów.

uzyskała nawet niższą wartość funkcji celu niż rozwiązanie zwrócone przez model CPLEX. Wynika to z faktu, że solver optymalizuje model wykorzystujący aproksymację odcinkowo-liniową funkcji celu, podczas gdy wartość raportowana w tabeli obliczana jest na podstawie dokładnej postaci funkcji.

Konieczność zastosowania takiej aproksymacji wynika z natury solvera. CPLEX jest solverem programowania liniowo-całkowitoliczbowego (MILP) — potrafi efektywnie rozwiązywać modele, w których zarówno funkcja celu, jak i ograniczenia są liniowe (z dopuszczeniem zmiennych binarnych), nie obsługuje natomiast bezpośrednio dowolnych funkcji nieliniowych. Tymczasem funkcja celu rozważanego problemu zawiera dwa składniki o charakterze osobliwym (hiperbolicznym): człon push $\tau^{(1)}_{mns} = b_n / y_{mns}$ oraz człon kolejkowy $\tau^{(2)}_l = 1/(C_l - \rho_l)$, gdzie ρ_l jest obciążeniem łącza. Oba mają postać $1/z$ i dążą do nieskończoności, gdy $z \to 0$ (przepustowość maleje do zera lub obciążenie zbliża się do pojemności łącza), więc nie można ich zapisać jako wyrażeń liniowych. Aby umieścić je w modelu MILP, każdą z tych funkcji zastępuje się funkcją odcinkowo-liniową, tj. łamaną opartą o zbiór punktów węzłowych (w implementacji odpowiednio 18 i 20 punktów), którą CPLEX potrafi obsłużyć. Dodatkowo iloczyny $x_{mns} y_{mns}$ (zmienna binarna razy ciągła) są linearyzowane standardową techniką *big-M*. Tak zbudowany model jest już liniowy i rozwiązywalny do optimum, jednak jego optimum dotyczy funkcji *przybliżonej*; różnica względem dokładnej funkcji celu jest źródłem obserwowanych ujemnych odchyleń, gdy heurystyka, oceniana

na dokładnej postaci Q, trafia w punkt o nieco niższej wartości rzeczywistej.

Największe różnice zaobserwowano dla grupy `two_big`. Dla wszystkich instancji tej grupy wartość funkcji celu uzyskana przez heurystykę była o około 41% wyższa od rozwiązania solvera. Wskazuje to na trudność heurystyki w efektywnym wykorzystaniu kilku alternatywnych ścieżek o wysokiej przepustowości.

Istotną obserwacją jest również czas działania obu metod, zilustrowany na rysunku 2. Dla najmniejszych instancji czasy działania były porównywalne, natomiast wraz ze wzrostem rozmiaru problemu czas obliczeń solvera rósł znacznie szybciej niż czas działania heurystyki. Dla przykładu, dla instancji `mixed_09` solver potrzebował ponad 17 sekund, podczas gdy heurystyka zakończyła działanie w czasie poniżej 2 sekund. Podobną zależność zaobserwowano dla pozostałych większych instancji.

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że heurystyka stanowi atrakcyjną alternatywę dla dokładnego rozwiązania optymalizacyjnego. Choć nie gwarantuje uzyskania optimum globalnego, w wielu przypadkach dostarcza rozwiązania o jakości zbliżonej do rozwiązania solvera przy znacznie krótszym czasie obliczeń i lepszej skalowalności.

6 Podsumowanie

W ramach pracy rozważono problem rozprzestrzeniania danych w sieci świadomej treści (CAN), polegający na jednoczesnym rozmieszczeniu obiektów na serwerach pamięci podręcznej, przypisaniu klientów do serwerów oraz alokacji przepustowości, przy celu minimalizacji sumy średnich opóźnień. Problem ten łączy część dyskretną (rozmieszczenie i przypisanie) z częścią nieliniową (alokacja przepustowości), przez co jest nieliniowym zadaniem programowania mieszanego, NP-trudnym już przy ustalonych przepustowościach.

Przygotowano i porównano dwa podejścia do jego rozwiązania. Pierwsze opiera się na dokładnym modelu rozwiązywanym solverem CPLEX, w którym osobliwe składniki funkcji celu zastąpiono aproksymacją odcinkowo-liniową, a iloczyny zmiennych zlinearyzowano techniką *big-M*. Drugie stanowi implementację dwupoziomowej heurystyki dekompozycyjnej zaproponowanej w pracy [1], która naprzemiennie rozwiązuje podproblem alokacji przepustowości (metodą SLSQP) oraz podproblem rozmieszczenia danych (zachłannie), startując z prostego rozwiązania dopuszczalnego.

Eksperymenty na czterech rodzinach instancji (`free`, `mixed`, `one_big`, `two_big`), różniących się profilem pojemności łączy, pozwoliły wyciągnąć następujące wnioski:

- heurystyka zawsze zwracała rozwiązanie dopuszczalne, a jakość tego rozwiązania silnie zależała od charakterystyki sieci — od praktycznie optymalnej (rodzina `one_big`) po odchylenie rzędu 40–50% (rodziny `free` i `two_big`);
- czas działania solvera rósł wraz z rozmiarem problemu znacznie szybciej

niż czas działania heurystyki, co czyni heurystykę istotnie lepiej skalowalną;

Podsumowując, dokładny solver jest wartościowym punktem odniesienia dla mniejszych instancji, w których dostarcza rozwiązania optymalnego, natomiast heurystyka dekompozycyjna stanowi praktyczną i dobrze skalowalną alternatywę, szczególnie dla sieci z wyraźnymi wąskimi gardłami. Dalsze prace mogły by obejmować implementację w pełni rozproszonej wersji algorytmu.

Literatura

[1] D. Gąsior, M. Drwal: *Decentralized Algorithm for Joint Data Placement and Rate Allocation in Content-Aware Networks*. W: A. Kwiecień, P. Gaj, P. Stera (red.), Computer Networks (CN 2012), Communications in Computer and Information Science, vol. 291, ss. 32–44. Springer, Berlin, Heidelberg (2012).